

Guía de trabajo en grupo

Producto de la Unidad 1 — Matriz de decisión tecnológica · Bases de Datos III

Cada sección trae un recordatorio corto y un ejemplo, si algo no les quedó claro en la explicación en vivo, esto les debería alcanzar para seguir solos.

Recordatorio rápido: familias y motores

MongoDB — documental: documentos tipo JSON sin esquema fijo. Sirve cuando los datos varían mucho entre registros (catálogos).

Redis — clave-valor: todo en memoria RAM, una clave que apunta a un valor. Sirve cuando la velocidad es crítica (cachés, sesiones).

Neo4j — grafos: nodos y relaciones guardadas como conexiones directas. Sirve para redes y recomendaciones.

Cassandra — columnar: datos distribuidos en muchos servidores. Sirve para volúmenes de escritura enormes (sensores, logs).

PostgreSQL — relacional (SQL): tablas fijas con filas y columnas. Sirve cuando la consistencia es innegociable (pagos, transacciones).

Nombres del grupo: Jackeline Orozco Echeverry, Lina Marcela Sepúlveda Henao y Sara Luz Pérez Moreno

Fecha: 20/09/2026

1. Caso real

Plantilla: "[Sistema] que maneja [tipo de datos] para [quién lo usa]". Tiene que ser específico, no sirve "una empresa" o "una app".

Ejemplo

"Plataforma de streaming de música que maneja catálogo de canciones, historial de reproducción y recomendaciones, para usuarios finales."

Si no tienen un caso propio, tomen uno del banco de 8 casos de respaldo.

Red social o foro universitario que maneja perfiles de usuario, publicaciones, sesiones activas y conexiones entre estudiantes, grupos de estudio y materias, para los estudiantes de una comunidad universitaria.

Caso 5 del banco de casos: estudiantes forman grupos de estudio, publican en foros y se conectan por intereses académicos.

2. Tabla comparativa de motores candidatos

Elijan UNA parte de su sistema (no todo a la vez). Para esa parte, elijan 2-3 motores candidatos y califiquen cada criterio como Alta/Media/Baja, con una justificación de una línea.

¿Qué pregunta cada criterio?

Consistencia — ¿qué tan grave es que el dato esté desactualizado por un instante? (grave, como en dinero → SQL; no grave, como un contador de likes → NoSQL puede.)

Latencia — ¿qué tan rápido necesito la respuesta: milisegundos, o puede tardar un poco más?

Escalabilidad — ¿cuánto va a crecer esto: cientos, miles, millones de registros?

Costo operativo — ¿cuánto cuesta mantener y operar ese motor a ese volumen?

Criterio	Motor 1: MongoDB (documental)	Motor 2: Redis (clave-valor)	Motor 3: Neo4j (grafos)
Modelo de datos	Documental (JSON/BSON)	Clave-valor	Grafos (nodos y relaciones)
Consistencia	Alta — fuerte por defecto, configurable a eventual	Media — eventual en clúster; muy rápida	Alta — transacciones ACID
Latencia	Media-baja — disco/SSD	Baja — en memoria, microsegundos	Media — depende de la profundidad de las consultas
Escalabilidad	Alta — horizontal (sharding), muy buena	Muy alta — horizontal, excelente	Media-alta — horizontal posible, pero más compleja
Costo operativo	Medio — requiere memoria para mantener buen rendimiento	Bajo — poco hardware, pero la RAM es cara	Medio-alto — las consultas complejas demandan recursos
Escalabilidad	Alta	Muy alta	Medio-alta

Aplicación al caso Red social o foro universitario

Motor	Qué guardaremos en cada una
MongoDB	Perfiles de usuario publicaciones- estructura flexible según el tipo de contenido (Preguntas, recursos, eventos)
Redis	Sesiones activas, notificaciones, pendientes, contador de cuántas veces entra en línea
Neo4j	Red de conexiones entre estudiantes, grupo de estudio y materias compartidas

Ejemplo de referencia (catálogo de canciones de una app de streaming) — la suya no tiene que verse igual, es solo para que vean cómo se llena:

Criterio	MongoDB (documental)	PostgreSQL (SQL)	Cassandra (columnar)
Consistencia	Baja — eventual, aceptable en un catálogo	Alta — más de lo que este caso exige	Media — eventual, ajustable
Latencia	Media — buena para lecturas frecuentes	Media-alta — JOINS con metadatos variables cuestan	Baja — pensada para escritura masiva
Costo operativo	Medio	Medio — paga funciones que aquí no se usan	Alto — más nodos para tolerancia a fallos

3. Justificación final

Plantilla: "Para _____, elegí _____ porque _____ es la prioridad, evidenciado por _____. " Tiene que citar el criterio Y una evidencia concreta de su caso — no basta con decir "uso MongoDB".

Ejemplo

"Para el catálogo de canciones, elegí MongoDB porque la variedad de metadatos es alta y cambia por género — un esquema fijo obligaría a columnas vacías o tablas separadas por tipo de contenido, evidenciado en que un track de jazz y uno electrónico casi no comparten atributos."

Motor 1 — MongoDB (documental)

Se necesitaría para los diversos tipos de publicaciones que hagan los estudiantes otro tipo de base de datos que soporten imágenes, videos, enlaces a otras direcciones URL's además para ello hemos pensado que se necesitaría una base de datos NO SQL para esta se ha pensado en MongoDB, ya que, permite guardar todo en una sola publicación sin tener que reconfigurar la base de datos cada vez que agregues un nuevo tipo de contenido, trabajar con MongoDB se siente muy parecido a manejar objetos normales en lenguajes como JavaScript o Python, se pueden añadir funciones nuevas sin dañar el código o afectar otras funcionalidades, MongoDB está diseñado de fábrica para repartir la carga de trabajo entre varios servidores sin complicaciones. Para que sea correcto y óptimo el uso de esta base de datos debemos asegurarnos de mantener el orden desde el código: por ejemplo, no se cree un comentario sin autor.

Colecciones y documentos

JSON



```
{
  "_id": ObjectId("123"),
  "nombre": "Carlos",
  "correo": "carlos@univ.edu"
}
```

Colección publicaciones (Incrustando comentarios): En lugar de crear una colección separada para comentarios, guardas las respuestas dentro del mismo documento del post.

JSON



```
{
  "_id": ObjectId("456"),
  "titulo": "Duda sobre Cálculo EDO",
  "contenido": "¿Alguien me ayuda con la regla del producto?",
  "categoria": "Ingeniería",
  "autor": {
    "usuario_id": ObjectId("123"),
    "nombre": "Carlos"
  },
  "comentarios": [
    {
      "usuario_id": ObjectId("789"),
      "nombre": "Ana",
      "texto": "Recuerda que  $u \cdot v' + u' \cdot v$ ",
      "fecha": "2026-09-17"
    }
  ]
}
```

Cómo se hacen las relaciones:

- Relación por Incrustación (Recomendada para comentarios): Los comentarios se guardan en un arreglo [] dentro del documento de la publicación. Al buscar el post, traes todo de un solo viaje sin hacer uniones.
- Relación por Referencia (Para el Autor): Guardas únicamente el ObjectId("123") del usuario dentro del post para conectar quién lo escribió sin duplicar todos los datos del perfil.

Motor 2 — Redis (clave-valor)

Para las sesiones activas, las notificaciones pendientes y el contador de usuarios en línea, elegimos Redis porque la latencia es la prioridad, evidenciado por que estos datos se consultan y cambian

constantemente y deben responder en microsegundos: Redis guarda todo en memoria como pares clave-valor. Además, no exige consistencia estricta (un contador de «en línea» puede estar desactualizado un instante). Su costo es la RAM, que es cara, por eso lo reservamos para datos temporales.

Motor 3 — Neo4j (grafos)

Para las conexiones y recomendaciones entre estudiantes, elegimos Neo4j porque las relaciones son la prioridad, evidenciado por consultas como saber si el estudiante A es amigo del estudiante B, qué grupos de estudio siguen en común o recomendar compañeros de la misma facultad.

Neo4j guarda la información como nodos (personas) y líneas de conexión (relaciones), enfocándose en cómo se conectan las cosas, y ofrece consistencia fuerte con transacciones ACID. Su costo es medio-alto porque las consultas complejas demandan recursos.

Complemento — SQL

En nuestra idea de desarrollar una Red social o foro de una comunidad universitaria es necesario utilizar diversos tipos de bases de datos, es importante que se tenga para la parte de ingreso de datos de registro de los estudiantes usuarios una base de datos SQL ya que ella conecta todo fácilmente: Un foro es como una cadena: alguien escribe una publicación, esa publicación recibe respuestas y todo se guarda en un tema específico. SQL fue diseñado exactamente para juntar todas esas piezas sin tener que repetir información.

Mantiene todo limpio si alguien se va: Si un estudiante borra su cuenta, SQL se encarga solo de borrar sus comentarios o ponerles la etiqueta de "usuario eliminado". Así no quedan mensajes "fantasma" que nadie sabe de quién son ni se arruina la página.

Evita errores y mantiene el orden: Funciona como un organizador estricto que no te deja guardar cosas incompletas. Por ejemplo, no permitirá que se publique un comentario "en el aire" sin saber a qué publicación pertenece, ni un mensaje sin título. con este tipo de bases de datos se podrá guardar la información de:

Usuarios y perfiles: Guardar datos fijos como nombre, correo institucional, contraseña y carrera.

Estructura del foro: Publicaciones, comentarios, categorías (ej. Ingeniería, Eventos, Avisos) y materias.

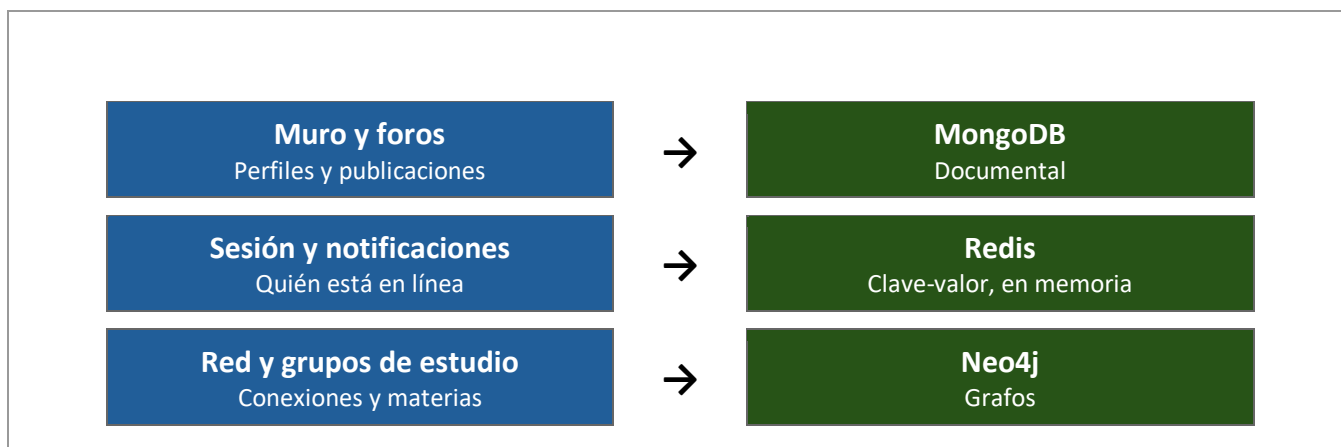
4. Arquitectura inicial

Pregunta guía: si tu aplicación tuviera 3 pantallas principales, ¿qué pantalla llama a qué base de datos? No necesitan UML formal, un diagrama de bloques simple basta.

Ejemplo

Pantalla de búsqueda → MongoDB (catálogo). Pantalla de reproductor → Redis (qué está sonando). Pantalla de recomendaciones → Neo4j (grafo de gustos). Tres cajas, tres flechas.

Pantalla de muro y foros → MongoDB (perfiles y publicaciones, con estructura flexible según el tipo de contenido). Pantalla de sesión y notificaciones → Redis (sesiones activas, notificaciones pendientes y contador de usuarios en línea). Pantalla de red y grupos de estudio → Neo4j (conexiones entre estudiantes, grupos de estudio y materias compartidas).



5. Repositorio Git

¿Por qué ahora, aunque no haya código? Ese primer commit ya cuenta como evidencia de trabajo desde el día uno — en la Unidad 8 van a necesitar el historial completo.

- ☐ Repositorio creado en GitHub
- ☐ git clone — traje el repositorio a mi computador
- ☐ git add . — marqué los archivos para guardar
- ☐ git commit -m "..." — guardé los cambios con un mensaje
- ☐ git push — subí los cambios a GitHub

URL del repositorio:
